

Strukture podataka i algoritmi

Grafovi

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

- Strukture podataka: Grafovi

„Matematičke strukture spadaju među najdivnija otkrića ljudskog uma. Njihova veličina je u njihovoj velikoj moći metafore i objašnjavanja realnosti“

Douglas Hofstadter

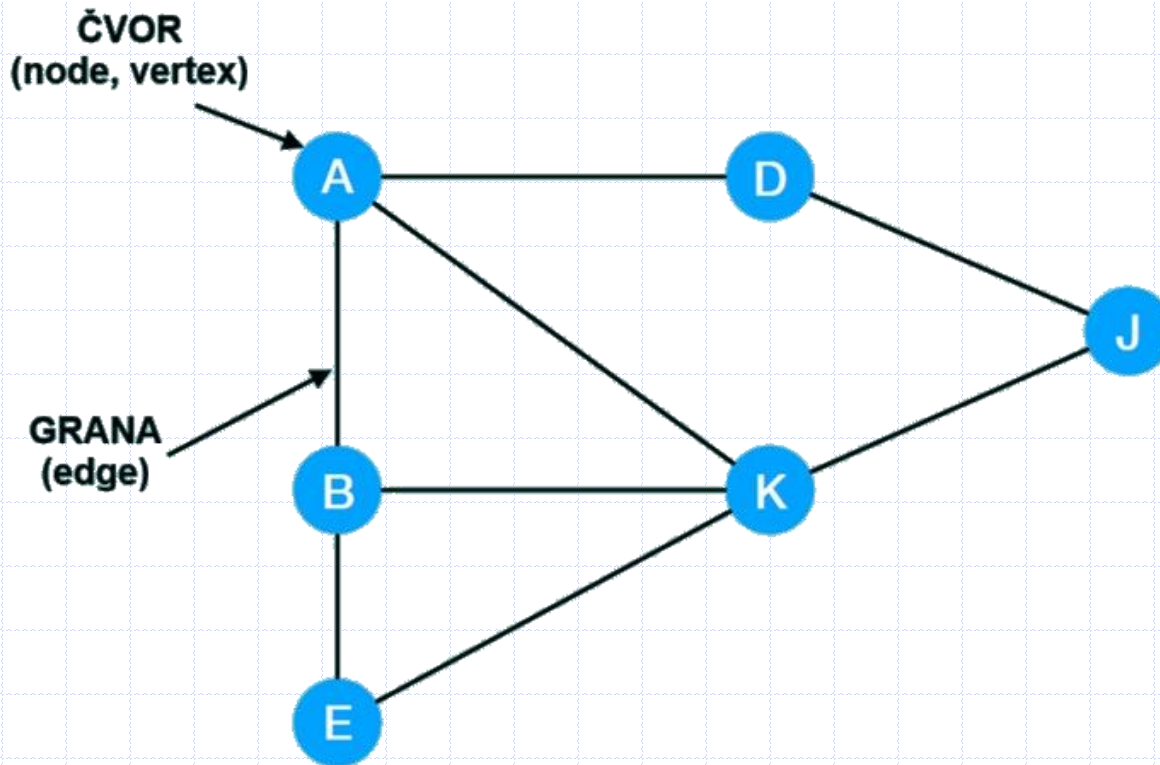
(Meta Mathematical Themas, 1985)

Grafovi

- **Grafovi** (graph) predstavljaju modele za različite strukture iz oblasti prirodnih, tehničkih ili društvenih nauka
- Primeri gde se grafovi koriste: saobraćajne mape, neuronske mreže, veštačka inteligencija, dizajn čipova, molekularne strukture, proučavanje migracija, epidemija i populacija
- Grafovi imaju primenu u mnogim oblastima, najčešće se prepoznaju pod nazivom mreže (transportne, komunikacione, računarske, socijalne mreže itd...)
- Zbog svoje široke praktične primene, grafovi su sveprisutni model podataka u računarstvu, a algoritmi grafova predstavljaju osnovu mnogih složenih algoritama koji se danas koriste
- **Teorija grafova** je deo matematike koja proučava grafove ili matematičke objekte koji modeluju binarne relacije među objektima

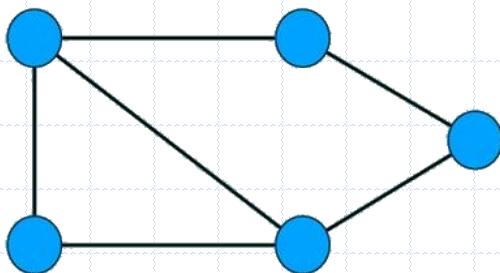
Grafovi

Graf se sastoji od konačnog skupa **čvorova** (teme, node, vertex, vertice) i skupa **grana** (edge) ili linija kojima su povezani parovi čvorova



Grafovi

Graf se sastoji od konačnog skupa **čvorova** (node, vertex) i skupa **grana** (edge) ili linija kojima su povezani parovi čvorova

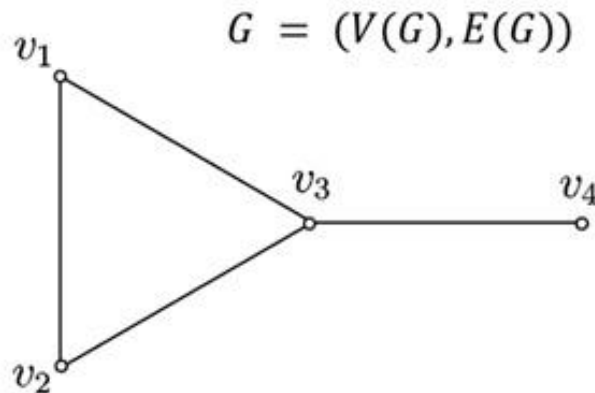


Grafovi

Definicija:

Graf G je uređen par $(V(G), E(G))$ gde je $V(G)$ konačan neprazan skup elemenata **čvorova**, a $E(G)$ je konačan skup neuređenih parova elemenata skupa $V(G)$ koji se zovu **grane**

Graf G se može geometrijski predstaviti crtežom u ravni, čvorovi grafa se predstavljaju tačkama ravni, a grane grafa linijama koje povezuju odgovarajuće čvorove

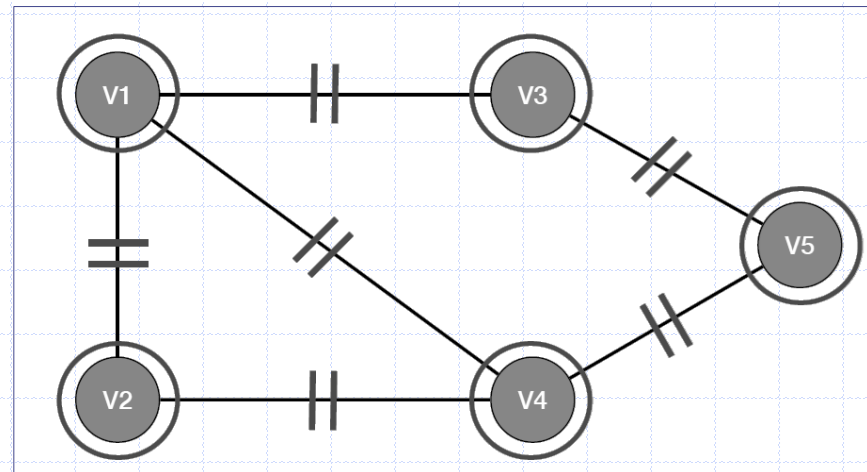


$G = (V(G), E(G))$

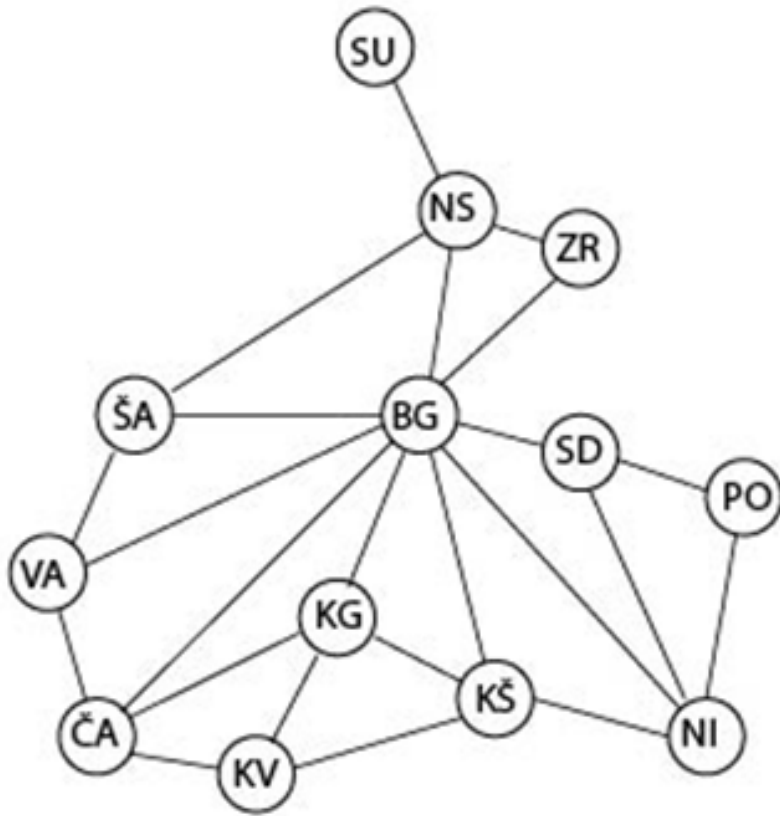
$$V(G) = \{v_1, v_2, v_3, v_4\}$$

$$E(G) = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_2, v_3\}, \{v_3, v_4\}\}$$

čvorovi i grane grafa



Grafovi - primer



Graf gradova u Srbiji povezanih putevima sastoji se od:

→ skupa V sa 13 čvorova:

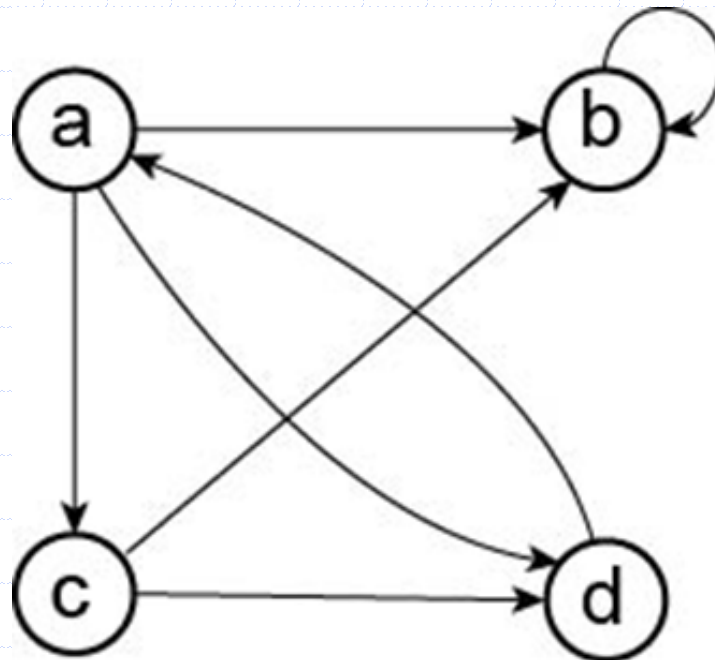
$V = \{BG, NS, NI, KG, SD, PO, VA, ČA, KV, KŠ, ŠA, ZR, SU\}$

→ skupa E sa 23 grane:

$E = \{\{BG, NS\}, \{BG, VA\}, \{BG, NI\}, \{KG, ČA\}, \dots\}$

- Graf se označava zapisom $G=(V, E)$ gde je:
 - V - skup čvorova
 - E - skup grana koje povezuju te čvorove
- Svaka grana $e \in E$ je dvoelementni podskup čvorova iz V koja spaja pojedine čvorove u i v iz V tj. $e = \{u, v\}$
- Za čvorove u i v koje spaja grana e kažemo da su susedni čvorovi

Grafovi - terminologija



Stepen čvora – predstavlja broj grana čvora

Grane (edges) – link između čvorova

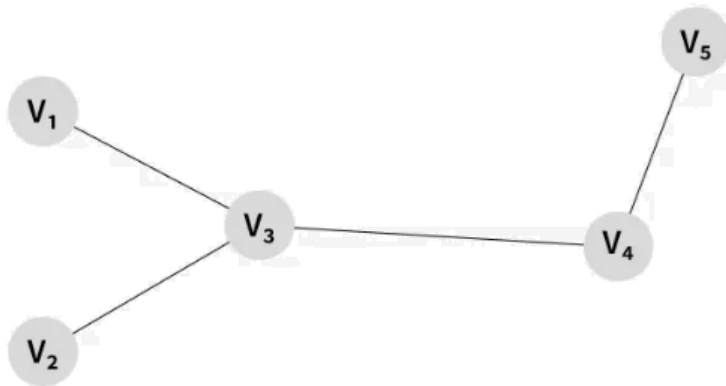
Susedni čvor – čvorovi na drugom kraju grane

Neusmeren graf – graf kod kojeg grane nemaju definisan smer

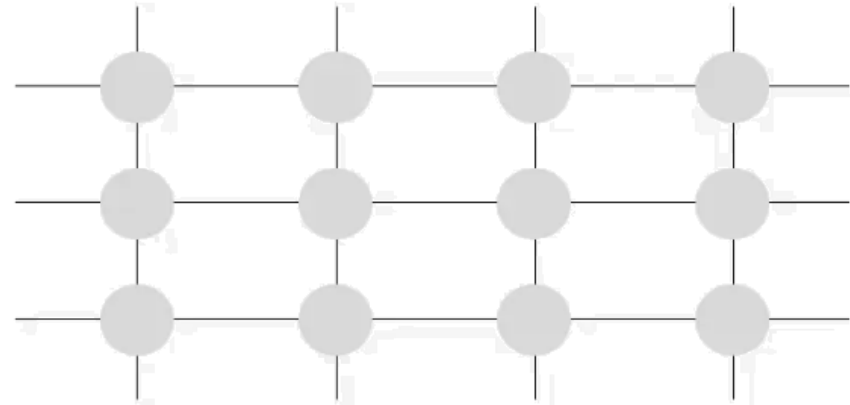
Usmeren graf – grane imaju oznaku za početak i kraj (strelica označava smer)

Petlja (loop) – usmerena grana koja ima početak i kraj u istom čvoru

Vrste grafova (prema veličini)



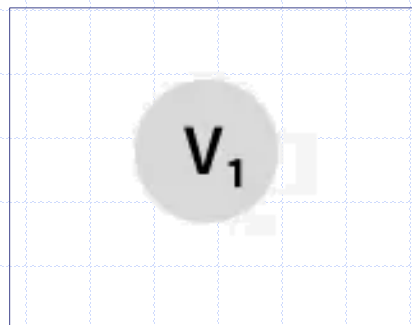
Konačni graf



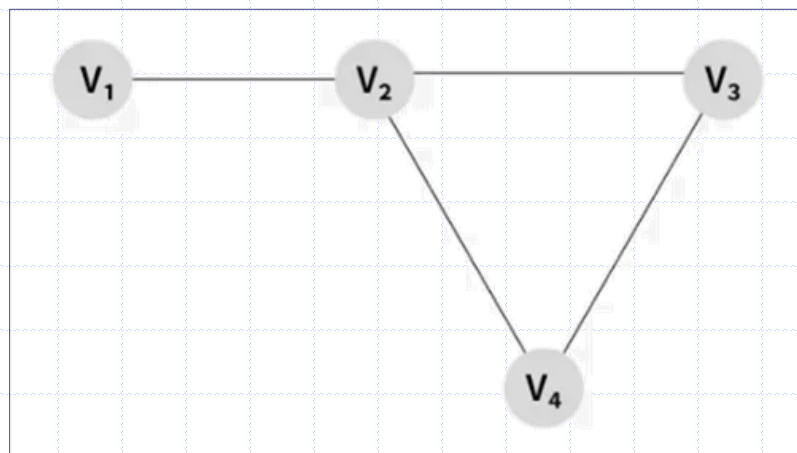
Beskonačni graf

- **Konačni graf:** Graf sa konačnim brojem čvorova i grana, broj čvorova i broj grana u konačnom grafu su ograničeni i mogu se prebrojati, koriste se za realne situacije gde postoji ograničen broj objekata i njihovih veza
- **Beskonačni graf:** Graf se naziva beskonačnim grafom ako ima beskonačan broj čvorova i beskonačan broj grana i ako se proteže beskonačno

Vrste grafova (prema strukturi)



Trivijalni graf

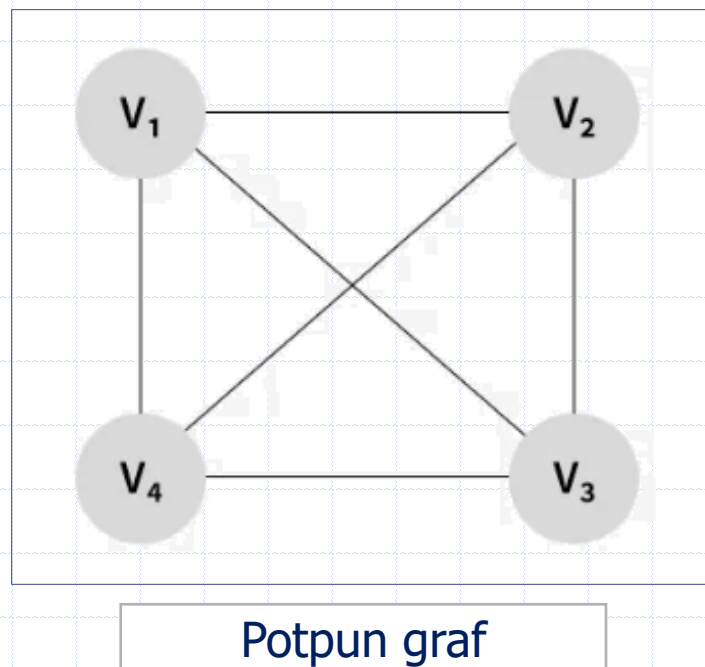
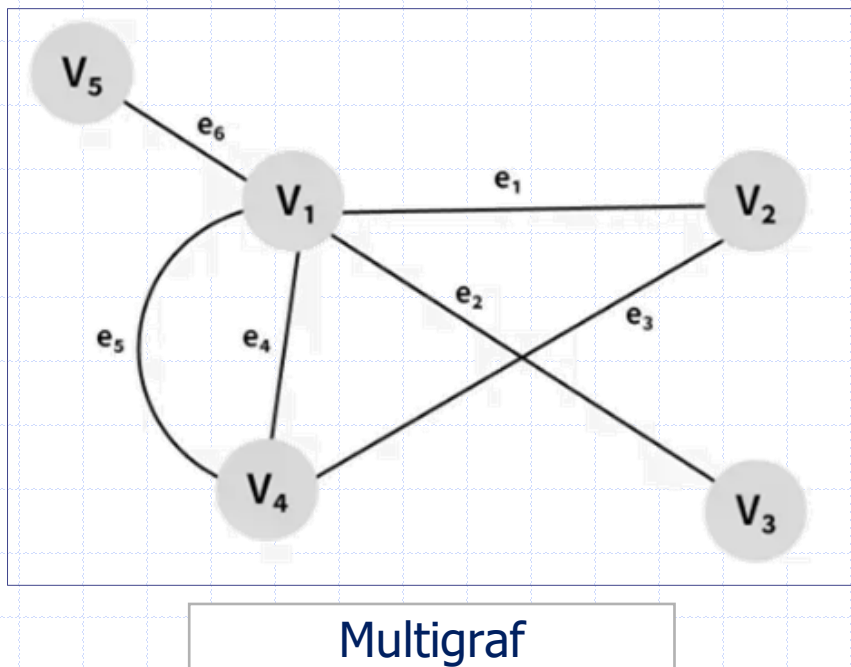


Prost graf

Trivijalni graf: Graf se naziva trivijalnim ako konačan graf sadrži samo jedan čvor i nijednu granu, naziv je i singletonski graf ili graf sa jednim čvorom, najjednostavniji tip grafa i polazna tačka za izgradnju složenijih grafova

Prost graf: Predstavlja graf koji ne sadrži paralelne grane niti petlje tj. nema više od jedne grane između para čvorova (železnička pruga koja povezuje različite gradove je primer jednostavnog grafa)

Vrste grafova (prema strukturi)



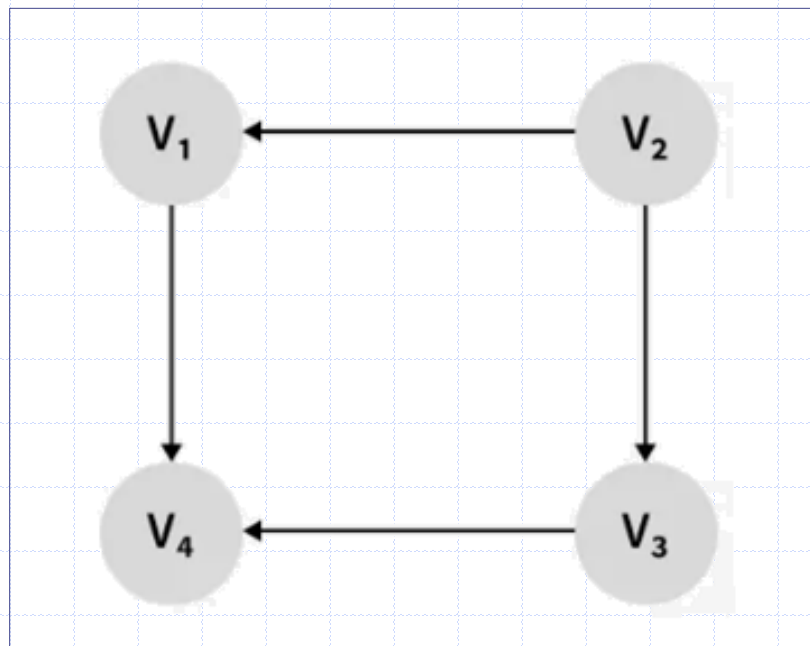
Multigraf: Graf koji sadrži pojedine paralelne grane, ali ne sadrži ni jednu petlju (npr. putna mapa)

Paralelne grane: ako su dva čvora povezana sa više od jedne grane, onda se takve grane nazivaju paralelnim (više ruta, ali jedno odredište)

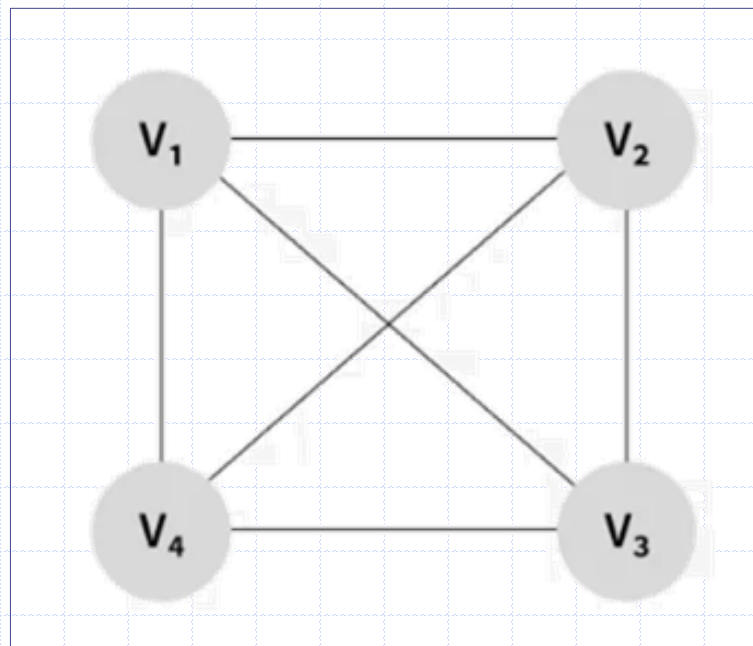
Petlja: grana grafa koja počinje iz čvora i završava se u istom čvoru

Potpun graf: Graf sa n čvorova naziva se potpun graf ako je stepen svakog čvora $n-1$, tj. jedan čvor je povezan sa $n-1$ grana

Vrste grafova (prema smeru)



Usmeren graf

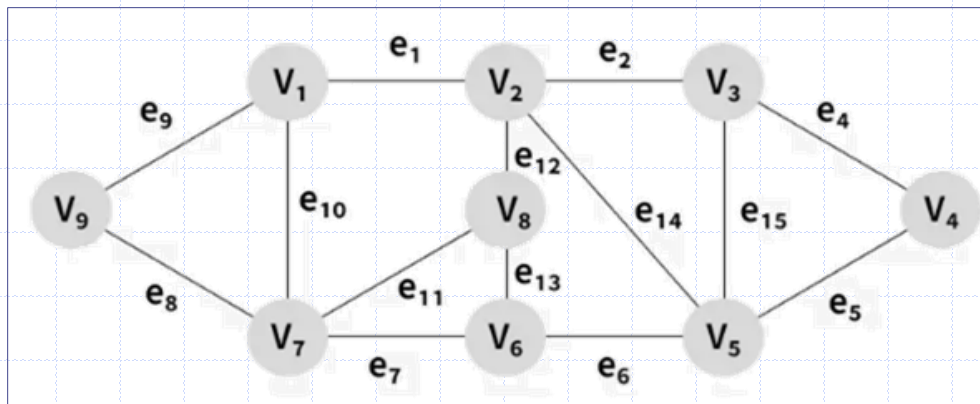


Neusmeren graf

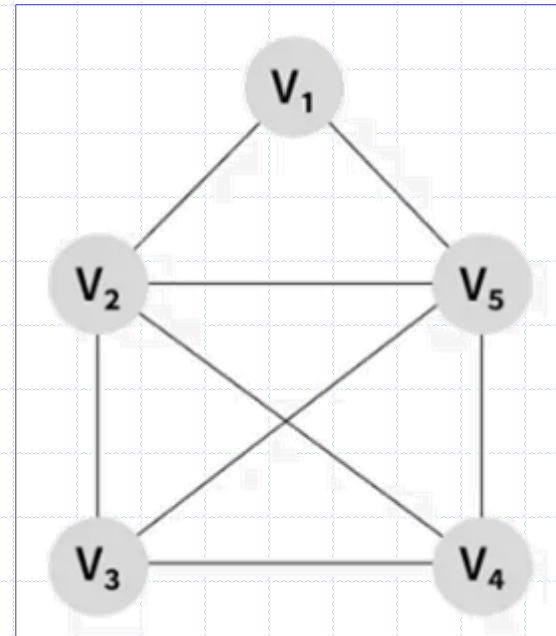
Usmeren graf: Graf koji sadrži grane sa definisanim smerom (strelice označavaju smer)

Neusmeren graf: Graf koji sadrži grane koje nemaju definisan smer (primer su friendship relacije na društvenim mrežama, dvosmerni putevi...)

Vrste grafova (prema težini)



Težinski graf

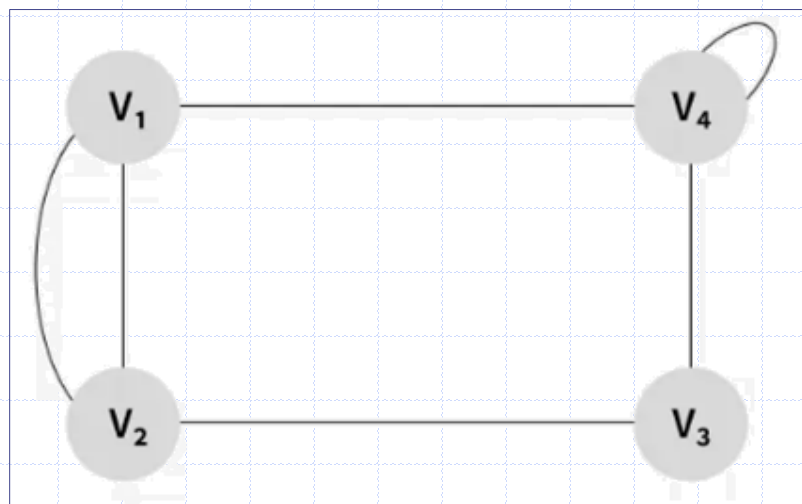


Beztežinski graf

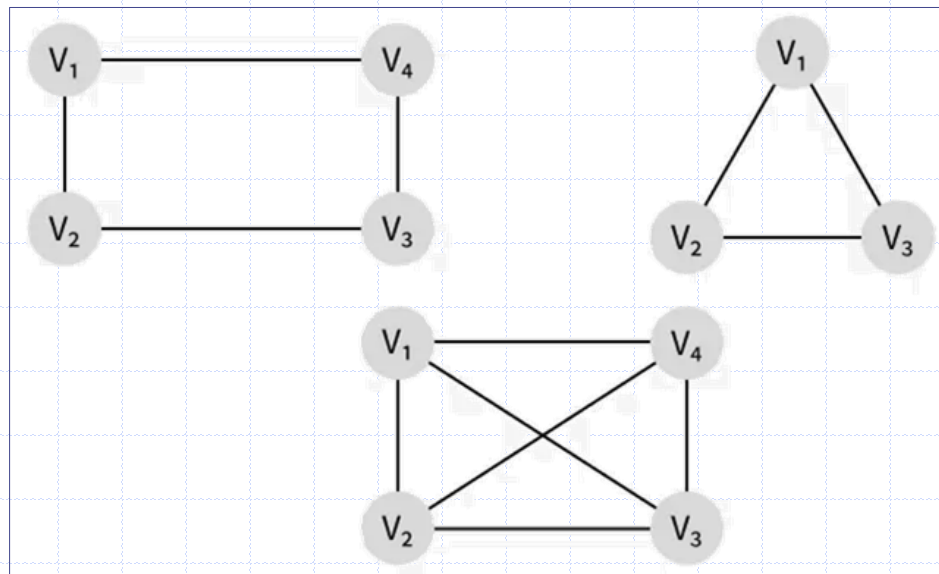
Težinski graf: Težinski ili ponderisani (weighted graph) sadrži grane koje imaju pridruženu težinu (udaljenost, cena, vreme...)

Beztežinski graf: Graf kod kojeg sve grane nemaju definisanu težinu ili se težina ne uzima u obzir, ovi grafovi se koriste za pronalaženje najkraćih ili najjeftinijih putanja ili ruta (primeri su Google mape, avio-rute i mreže za dostavu...)

Posebne vrste grafova



Pseudograf

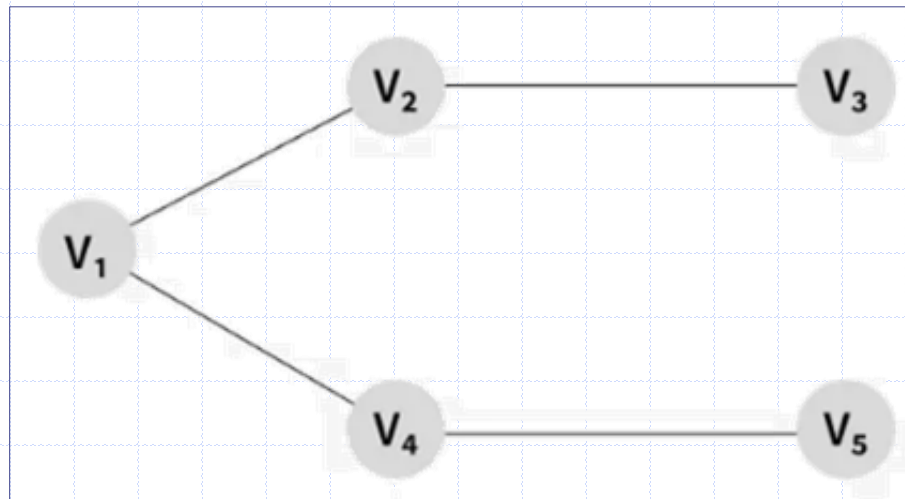


Regularan graf

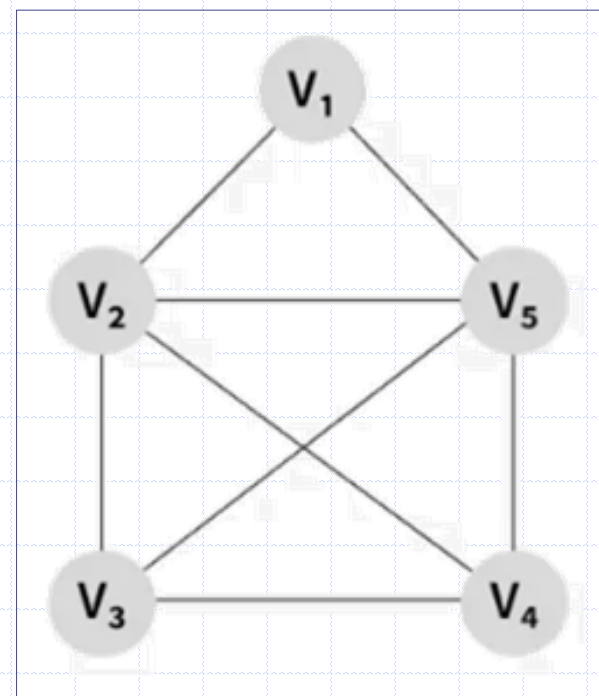
Pseudograf: Pseudograf može da sadrži i paralelne grane i petlje

Regularan graf: Neusmereni graf kod kojeg svi čvorovi imaju isti stepen, tj. isti broj grana ili suseda

Vrste grafova (prema osobinama)



Sparse Graph

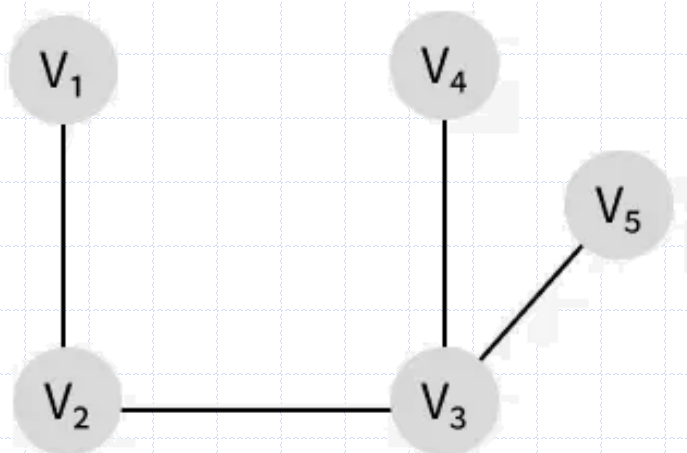


Dense Graph

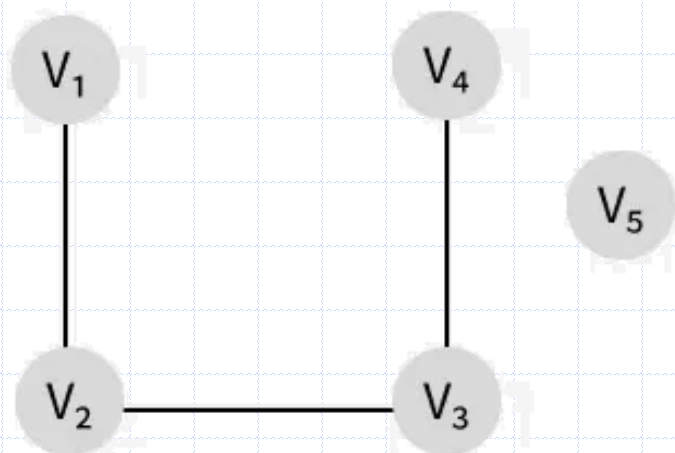
Sparse graf: Graf sa relativno malo grana u poređenju sa brojem čvorova (primer: grafik hemijskih reakcija gde svaki čvor predstavlja hemijsko jedinjenje, a svaka grana je reakcija između dva jedinjenja)

Dense graf: Graf sa mnogo grana u poređenju sa brojem čvorova (primer: grafovi društvenih mreža gde svaki čvor predstavlja osobu, a svaka grana je „prijateljstvo“)

Vrste grafova (prema povezanosti)



Povezan graf



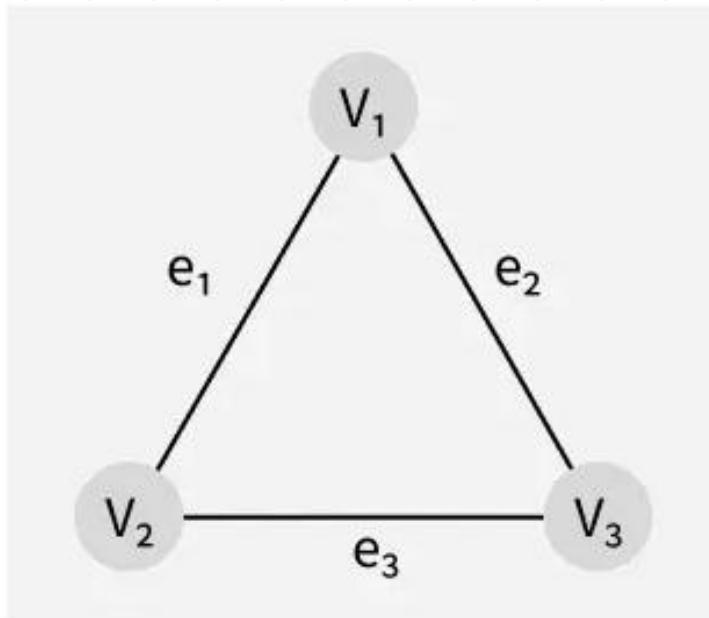
Nepovezan graf

Povezan graf: Graf kod kojeg postoji put (grana) između svaka dva čvora

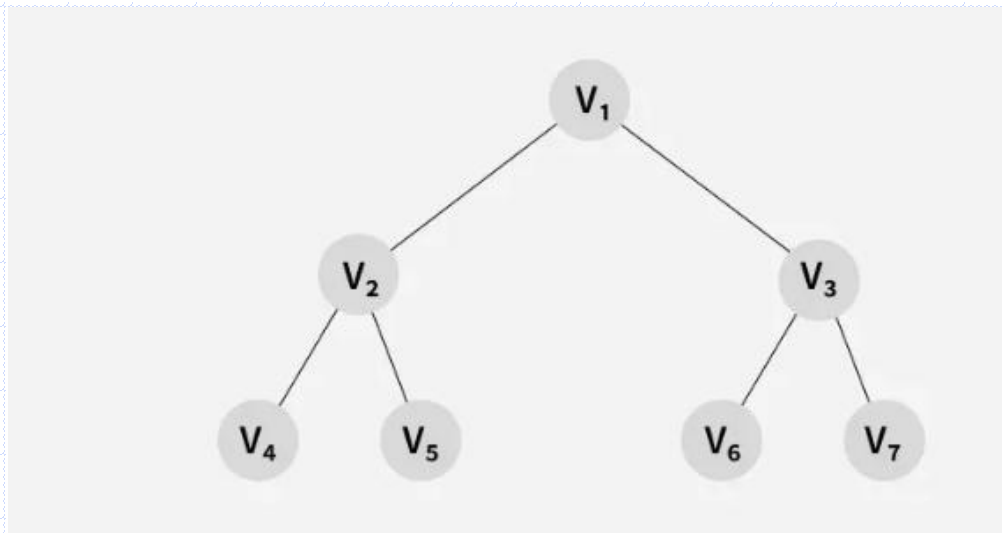
Nepovezan graf: Graf kod kojeg ne postoje definisane grane između pojedinih čvorova

Nulti graf: Graf sa n čvorova je nepovezan graf koji se sastoji od n komponenti i svaka komponenta se sastoji od jednog čvora bez grana

Vrste grafova (prema formi)



Ciklični graf

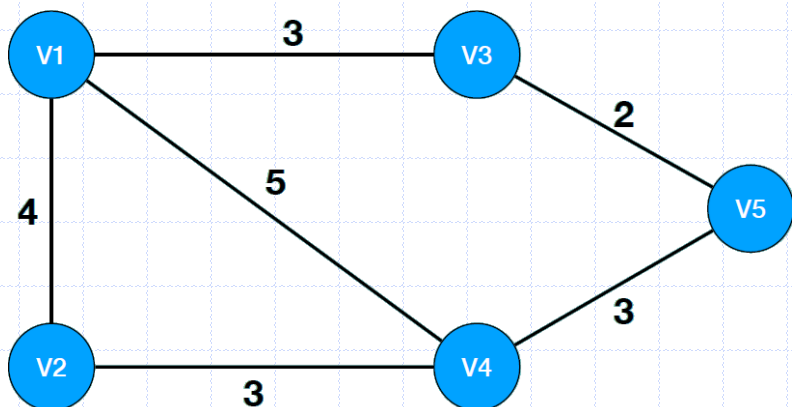


Stablo

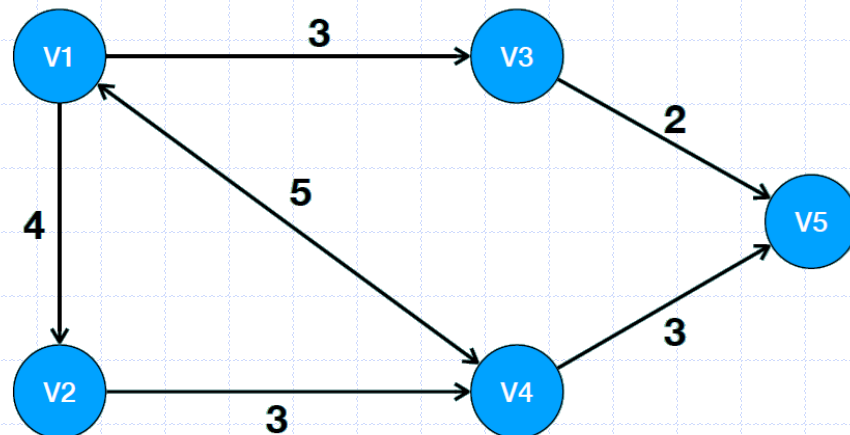
Ciklični graf: Graf G koji ima cikluse, sastoji se od n čvorova i $n \geq 3$, tj. $V_1, V_2, V_3 \dots V_n$ i grana $(V_1, V_2), (V_2, V_3), (V_3, V_4) \dots (V_n, V_1)$, ovaj graf ima najmanje jednu petlju

Stablo: Graf u kome su svaka dva čvora povezana tačno jednom granom, tj. svaki povezan aciklični graf je stablo

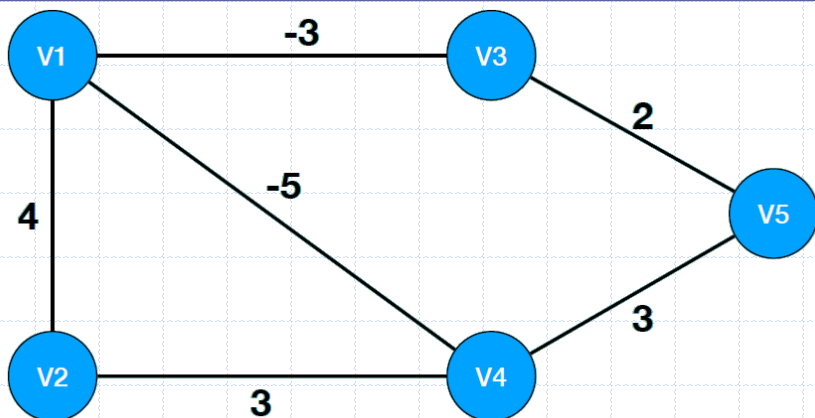
Vrste grafova (prema osobinama)



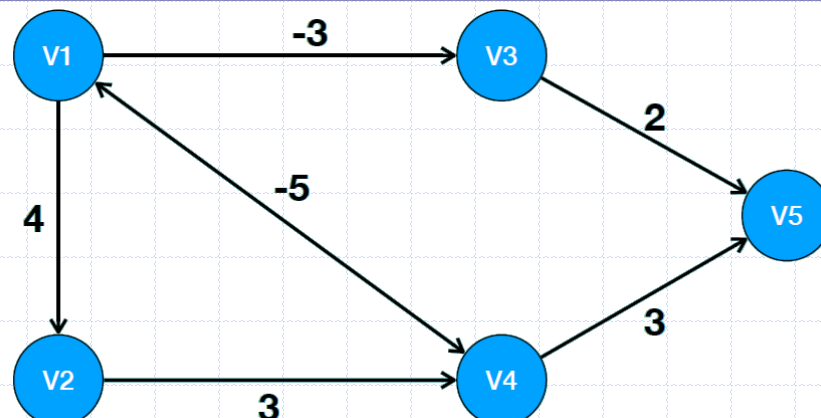
Težinski pozitivni neusmereni graf



Težinski pozitivni usmereni graf



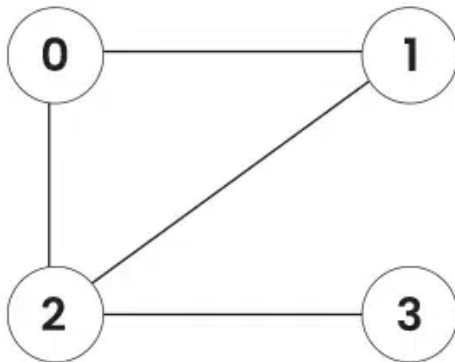
Težinski negativni neusmereni graf



Težinski negativni usmereni graf

Predstavljanje grafova

- Da bi se graf mogao efikasno koristiti u računarskim programima, neophodno je pronaći način njegovog predstavljanja koji omogućava jednostavno manipulisanje čvorovima i granama
- Najčešće se koriste dva pristupa:
 - 1) **matrica susednosti** (Adjacency Matrix)
 - 2) **lista susednosti** (Adjacency List)
- Primer matrice susednosti:

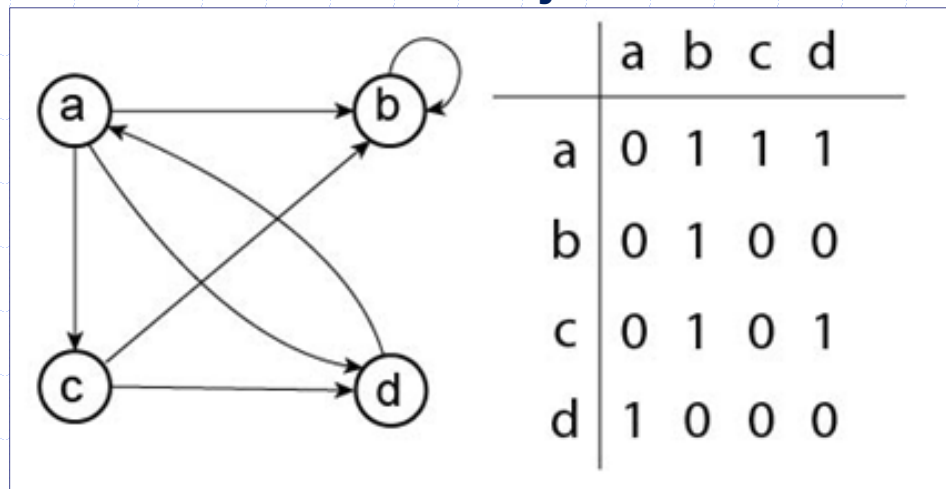


	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

$\text{matrica}[i][j] = 1$
ako postoji grana između
čvora **i** i čvora **j**
 $\text{matrica}[i][j] = 0$
ako nema grane između
čvora **i** i čvora **j**

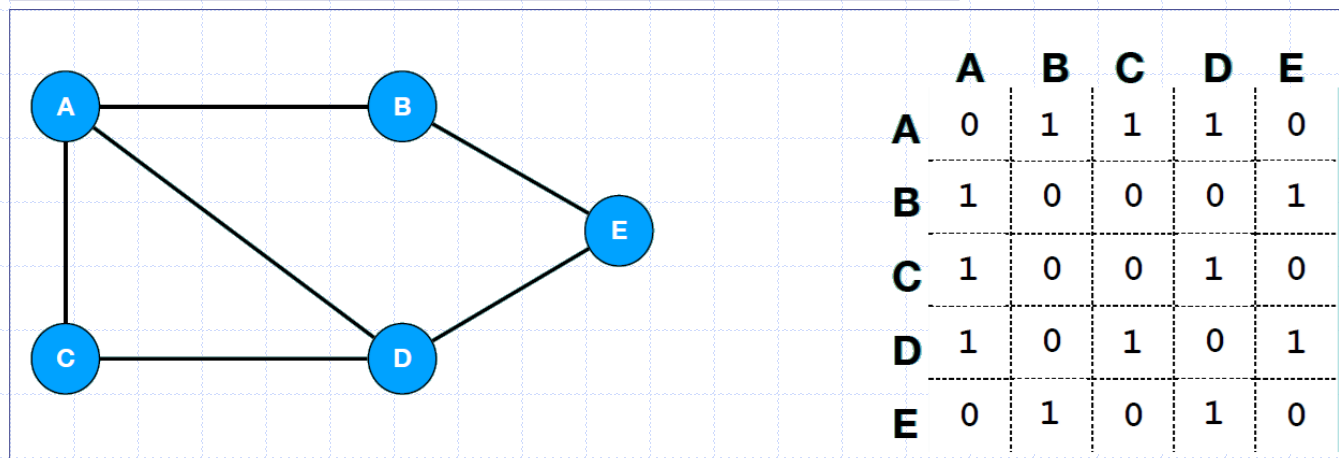
Matrica susednosti

- Matrica susednosti je 2D niz, čiji su broj redova i kolona jednaki broju čvorova grafa, to je kvadratna matrica reda n (gde je n broj čvorova grafa), element matrice a_{ij} jednak je 1 ako postoji usmerena grana između čvorova v_i i v_j , u suprotnom je 0



Matrica susednosti
za usmeren graf

Matrica susednosti
za neusmeren graf

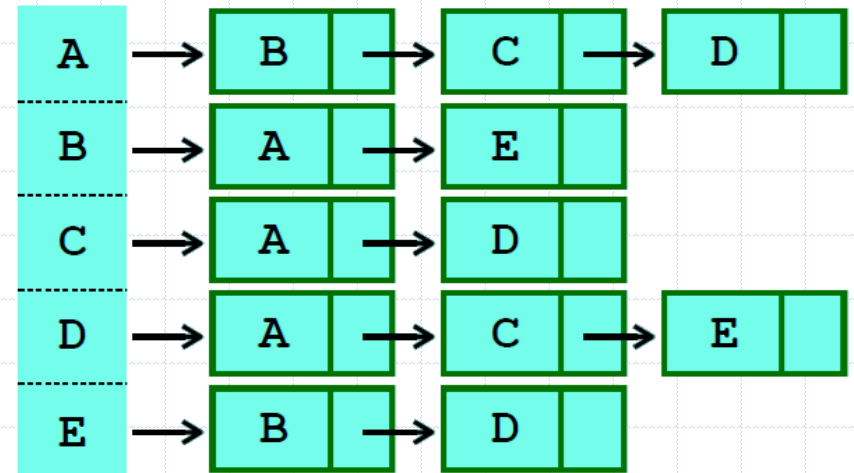
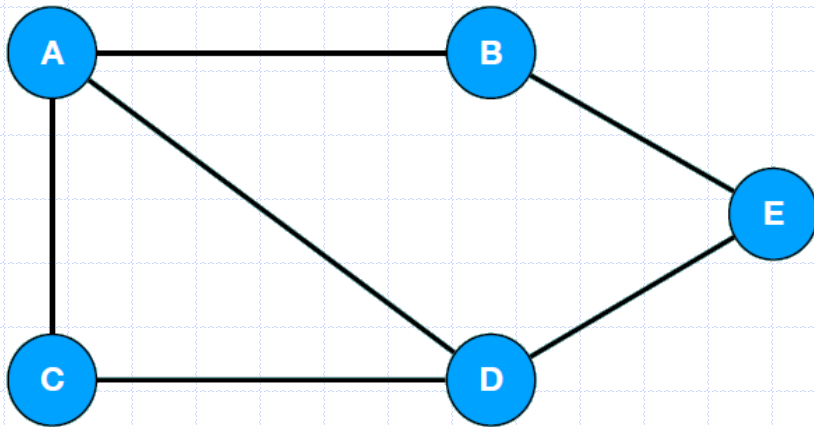


Matrica susednosti

- Kod neusmerenih grafova matrica je simetrična, što znači da je $a_{i,j} = a_{j,i}$ za svako i i j , tako da se jedna neusmerena grana između dva čvora predstavlja dvema usmerenim granama, jednom granom od prvog do drugog čvora i jednom granom od drugog do prvog čvora
- Prednost ovog pristupa je jednostavnost jer je vreme pristupa bilo kom elementu matrice konstantno, odnosno to vreme ne zavisi od broja čvorova i grana grafa.
- Ovim pristupom lako je proveriti da li su dva čvora povezana, kao i dodati ili obrisati granu, pa se matrice susednosti često koriste u algoritmima gde je potrebno brzo proveriti postojanje veze između čvorova
- Nedostatak je što matrica uvek zauzima prostor reda n^2 , što može biti neefikasno kod grafova sa malim brojem grana

Lista susednosti

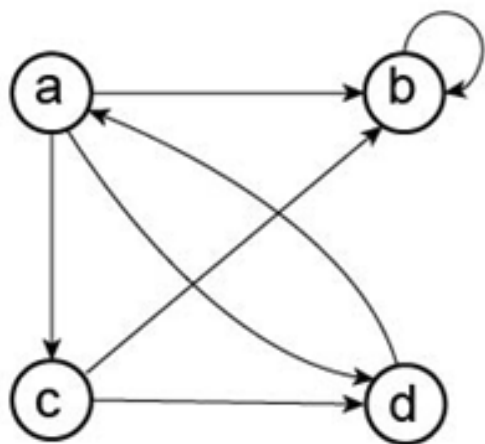
- Kod liste susednosti svakom čvoru grafa pridružuje se lista čvorova do kojih vodi grana iz tog čvora, npr. ako iz čvora **A** postoje grane ka čvorovima **B** i **D**, lista susednosti za čvor **A** glasi: **A**→{**B,D**}
- Graf se predstavlja kao niz lista, gde svaki element niza predstavlja jedan čvor, a njegova lista sadrži čvorove susede, na taj način mogu se pronaći svi čvorovi koji su direktno povezani sa nekim čvorom



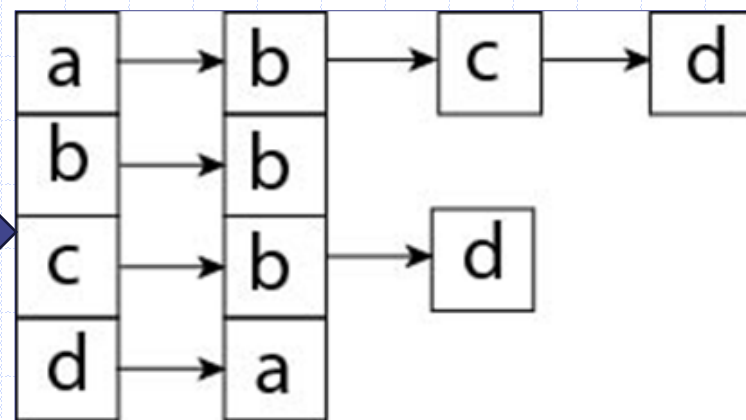
Lista susednosti za neusmeren graf

Lista susednosti

- Lista susednosti u praksi se može implementirati na više načina, pomoću povezanih lista, dinamičkih nizova, binarnih stabala ili heš tabela
- U situacijama kada je graf statičan (kada se ne dodaju i ne brišu čvorovi ili grane), moguće je graf prikazati i pomoću jednog statičkog niza koji kombinuje sve liste susednosti u jednoj strukturi



	a	b	c	d
a	0	1	1	1
b	0	1	0	0
c	0	1	0	1
d	1	0	0	0



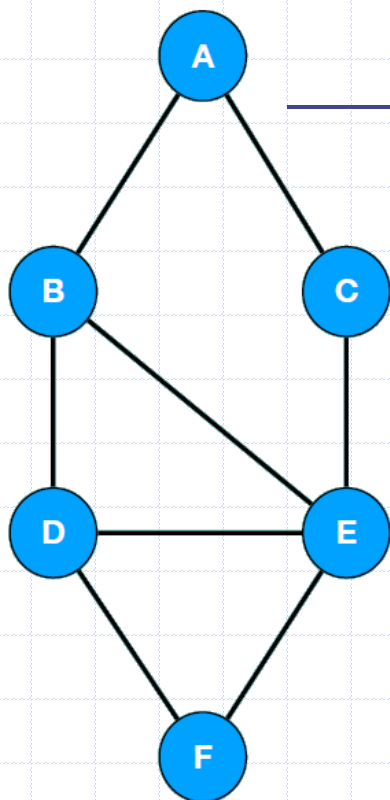
Matrica i lista susednosti

- Prednost liste susednosti je što zauzima mnogo manje memorije od matrice susednosti, posebno kada graf ima malo grana
- Njihova memorijska složenost je $O(|V| + |E|)$ za razliku od matrica susednosti čija je memorijska složenost $O(|V|^2)$ gde je $|V|$ broj čvorova, a $|E|$ broj grana grafa
- Operacije dodavanja i brisanja grana su jednostavne i efikasne
- Potreban prostor za predstavljanje grafa matricom susedstva i listom susedstva, za graf koji ima n čvorova i m grana je:

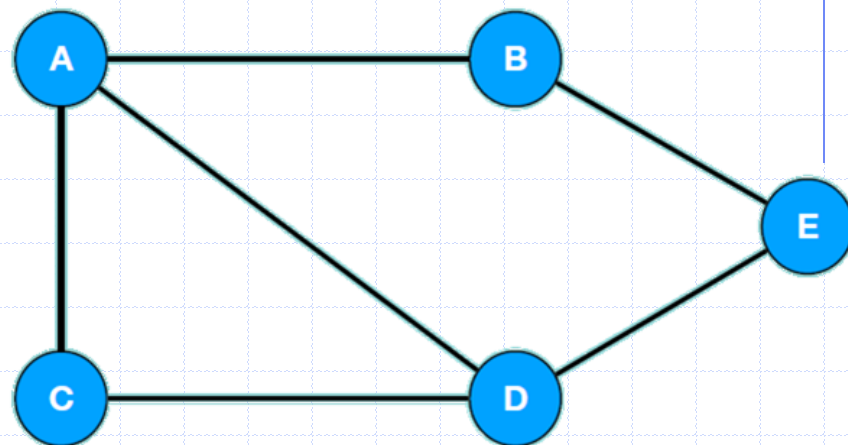
	matrica	lista
usmereni	n^2	$n + m$
neusmereni	$\frac{n(n + 1)}{2}$	$n + 2m$

Predstavljanje grafova

- Upotrebom Python REČNIKA



{ A : [B, C],
B : [A, D, E],
C : [A, E],
D : [B, E, F],
E : [C, D, F],
F : [D, E] }



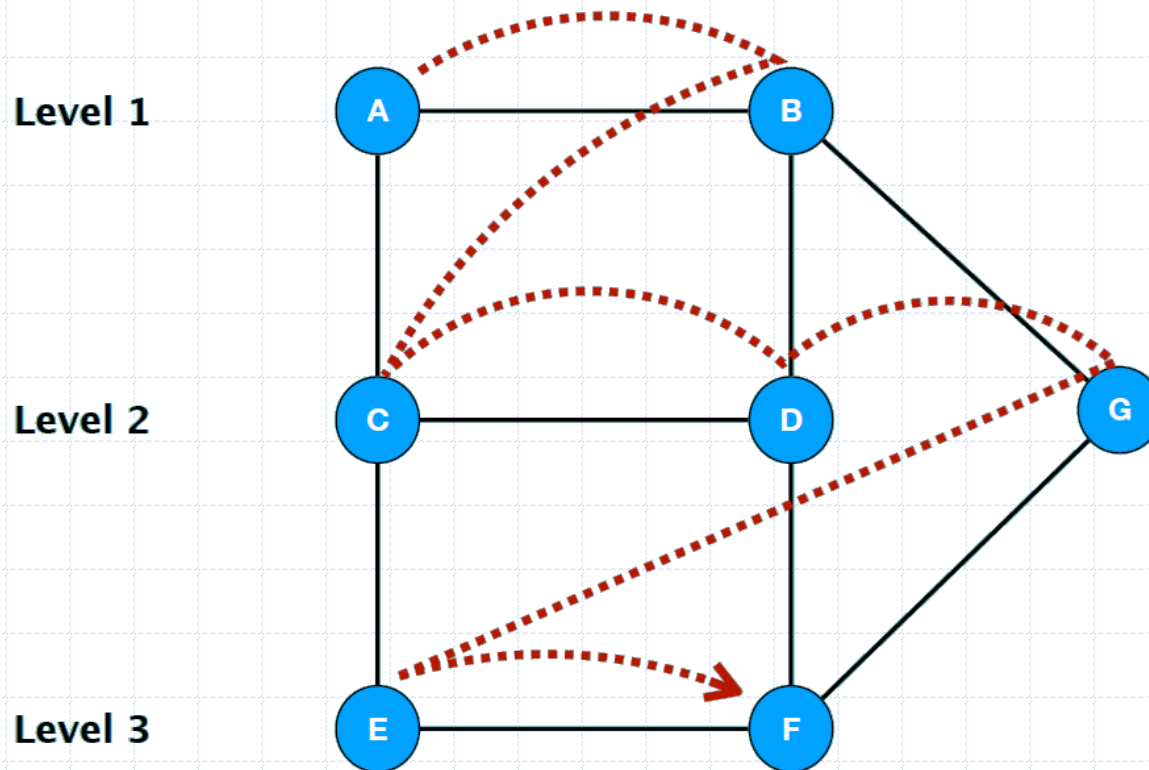
{ A : [B, C, D],
B : [A, E],
C : [A, D],
D : [A, C, E],
E : [B, D] }

Algoritmi obilaska grafova

- Izazovi u kreiranju algoritama za **obilazak grafova** (graph traversal)
- Kod nizova i skupova elementi se mogu pregledati linearnim redosledom, dok su grafovi složenije strukture pa je njihov obilazak zahtevnija procedura
- Za graf **G** i njegova dva čvora **a** i **b**, problem **a-b** povezanosti sastoji se u pronalaženju odgovarajućeg algoritma koji odgovara na pitanje „**da li postoji put između ta dva čvora**”
- U mnogim primenama grafova potrebno je pronaći put između dva čvora koji ima najmanje skokova, što znači najkraći put između dve tačke, pa se uvodi pojam rastojanja između dva čvora u grafu koji se definiše kao najmanje rastojanje između njih
- Postoje dva osnovna pristupa obilasku (pretraživanju) grafa:
 - a) pretraga u širinu **BFS** (Breadth First Search)
 - b) pretraga u dubinu **DFS** (Depth First Search)
- Neki od najvažnijih savremenih algoritama koji se zasnivaju ba BFS pretraži grafova su **Dijkstra**, **Kanov** i **Primov algoritam**

BFS obilazak grafa

- BFS je algoritam za obilazak strukture podataka grafa koji počinje na nekom proizvoljnom čvoru grafa i prvo se obilaze susedni čvorovi (koji su na trenutnom nivou) pre nego što pređe na susedne čvorove sledećeg nivoa (to čini prvi talas pretrage). Zatim se iz svakog od tih čvorova posećuju novi, neposećeni susedni čvorovi što čini sledeći talas. Proces se ponavlja sve dok ne postoje novi čvorovi koje je moguće otkriti



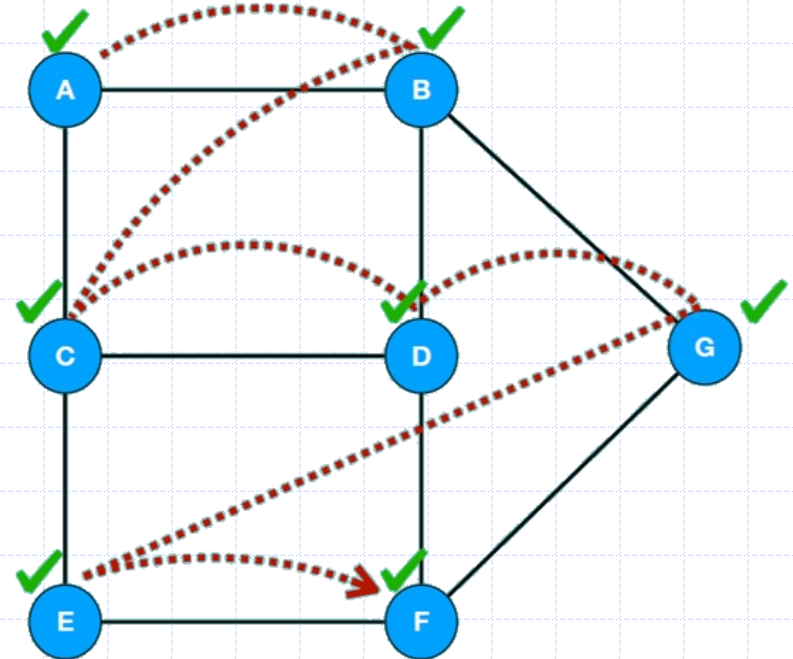
Algoritam počinje od izvornog čvora i istražuje sve čvorove dostupne od njega, posećujući čvorove u rastućem redosledu njihove udaljenosti od izvora, nivo po nivo koristeći red. Pošto grafovi mogu da sadrže cikluse, čvor se može posetiti više puta. Da bi se sprečilo ponovno posećivanje čvora, koristi se niz posećenih čvorova.

BFS obilazak grafa

BFS

```
enqueue any starting vertex  
while queue is not empty  
  p = dequeue()  
  if p is unvisited  
    mark it visited  
    enqueue all adjacent  
    unvisited vertices of p
```

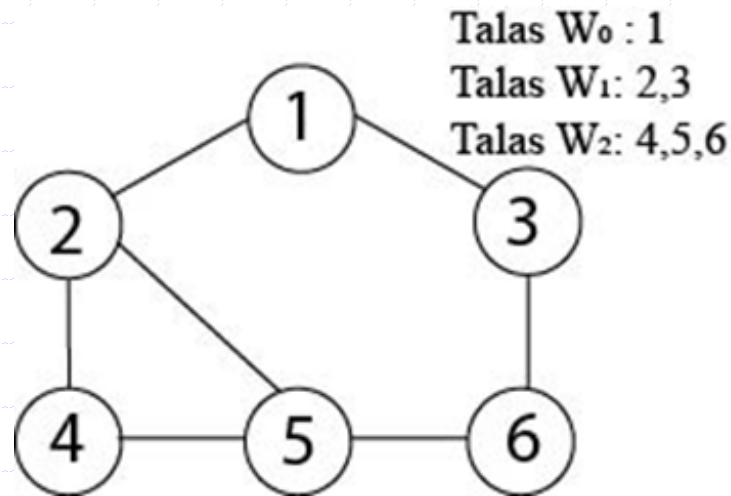
A B C D G E F



Queue



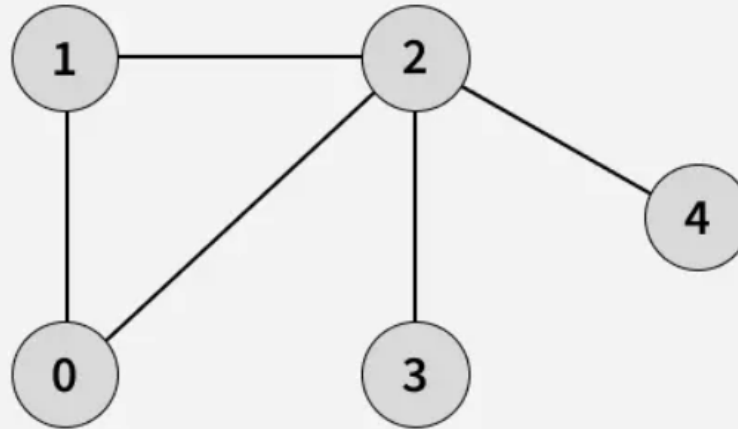
BFS obilzak grafa



- Čvorovi koji nisu obuhvaćeni nijednim talasom nisu dostupni iz početnog čvora i do njih ne postoji put u grafu
- Pretraga u širinu osim što pronalazi sve čvorove koji su dostupni od početnog, određuje i najkraće puteve do njih (po broju grana)
- Pretraga u širinu koristi red kao strukturu podataka, algoritam započinje dodavanjem početnog čvora u red, a zatim se iz reda uzimaju čvorovi redom, obilaze njihovi susedi i svaki novi neposećeni čvor se dodaje na kraj reda

BFS obilazak grafa

Input: $adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$



Objašnjenje:

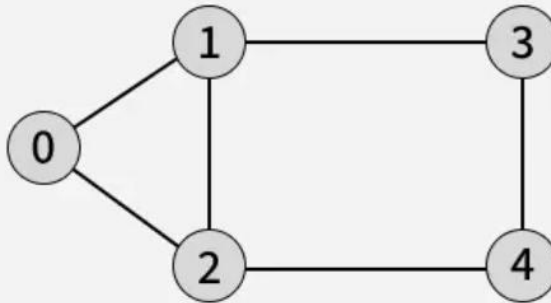
Počevši od 0, BFS obilazak se odvija na sledeći način:

- Poseta 0 - Ispis: 0
- Poseta 1 (sused od 0) - Ispis: 1
- Poseta 2 (sledeći sused od 0) - Ispis: 2
- Poseta 3 (prvi sused od 2 koji još nije posećen) - Ispis: 3
- Poseta 4 (sledeći sused od 2) - Ispis: 4

BFS obilazak grafa

01
Step

Maintain a visited array and a queue that keeps track of nodes to be processed.



visited[] =

--	--	--	--	--

res[] =

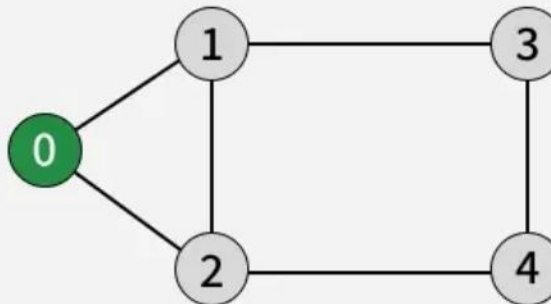
--	--	--	--	--

queue:

--	--	--	--	--

02

Start with source vertex 0 and push it into the queue



visited[] =

T				
---	--	--	--	--

res[] =

0				
---	--	--	--	--

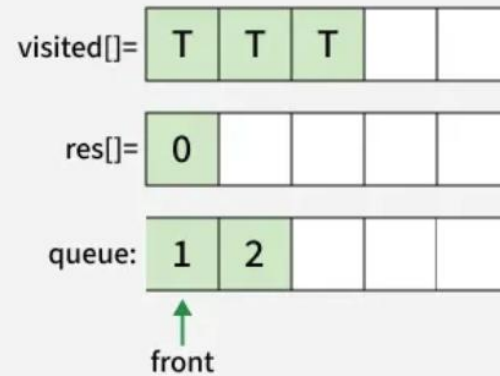
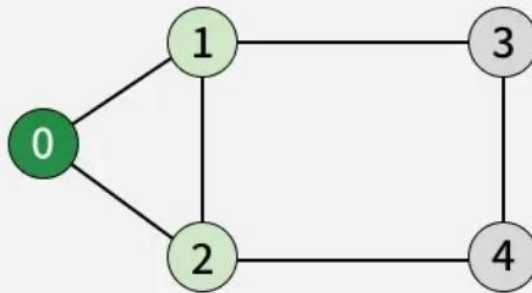
queue:

0				
---	--	--	--	--

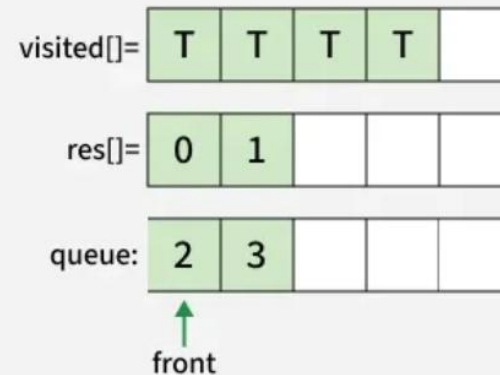
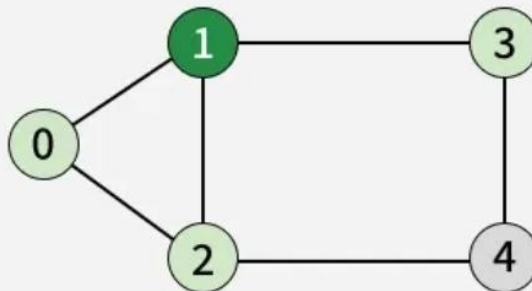
↑
front

BFS obilazak grafa

03 Process node 0, and push its unvisited neighbours into the queue.

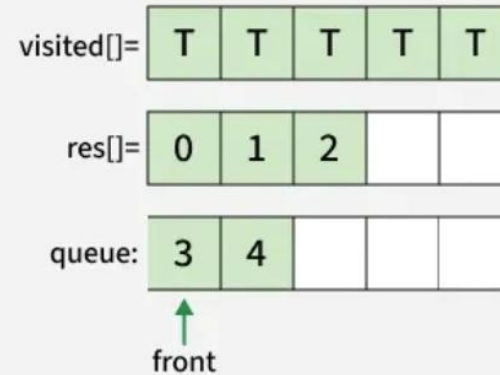
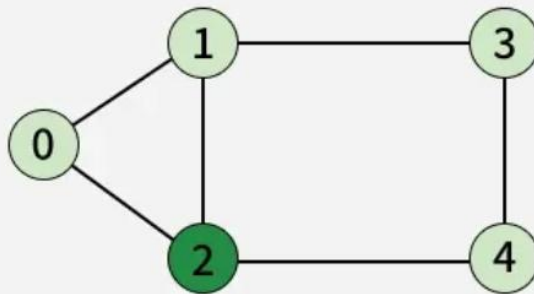


04 Process node 1, and push its unvisited neighbours into the queue.

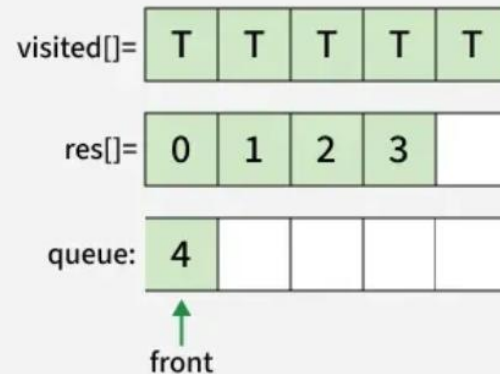
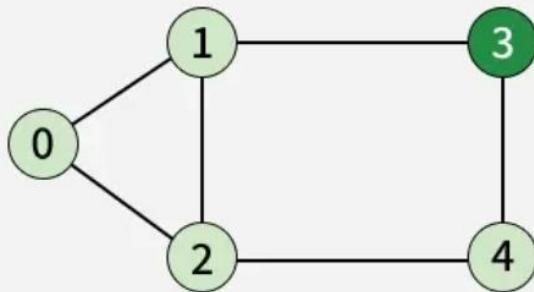


BFS obilazak grafa

05 Process node 2, and push its unvisited neighbours into the queue.

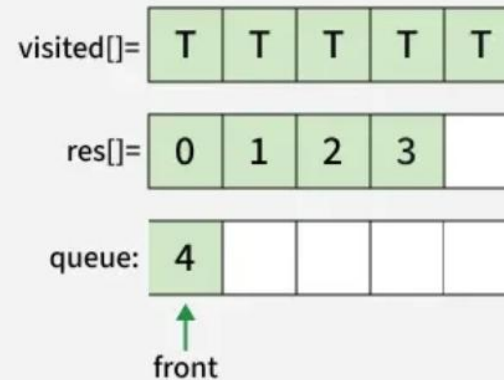
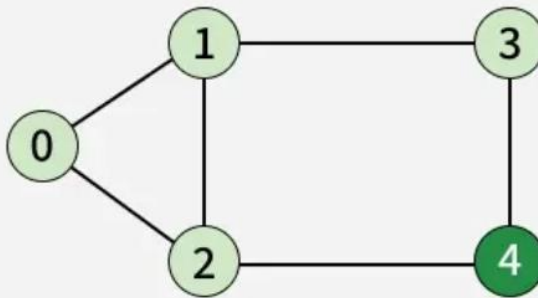


06 Process node 3, and push its unvisited neighbours into the queue.

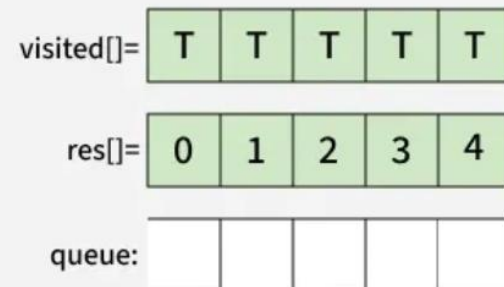
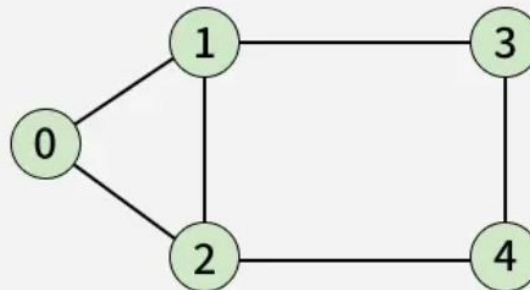


BFS obilazak grafa

07 Process node 4, and push its unvisited neighbours into the queue.



08 All the neighbours of 4 have been visited, proceed to the next node in the queue. Now that the queue is empty and there are no more nodes to process, we obtain the final BFS traversal.



BFS traversal order is 0, 1, 2, 3, 4.

BFS obilazak grafa – programski kôd

```
1  from collections import deque
2
3  # BFS za neusmeren graf
4  def bfs(adj):
5      V = len(adj)
6      visited = [False] * V
7      res = []
8
9      src = 0
10     q = deque()
11     visited[src] = True
12     q.append(src)
13
14     while q:
15         curr = q.popleft()
16         res.append(curr)
17
18         # obilazak svih neposećenih
19         # suseda tekućeg čvora
20         for x in adj[curr]:
21             if not visited[x]:
22                 visited[x] = True
23                 q.append(x)
24     return res
25
26 def addEdge(adj, u, v):
27     adj[u].append(v)
28     adj[v].append(u)
29
```

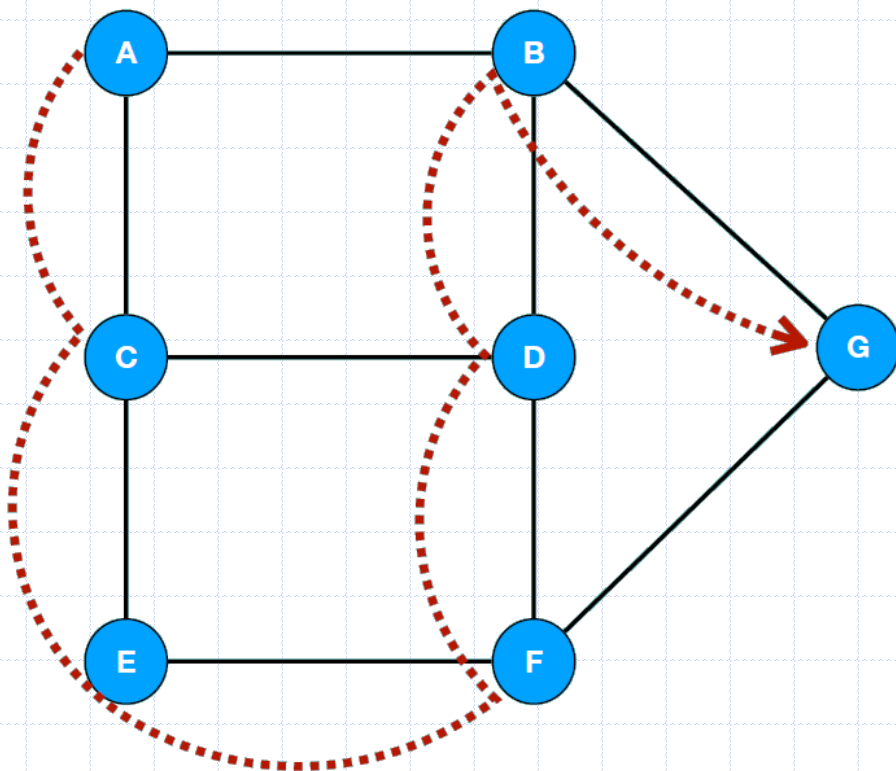
```
30
31 if __name__ == "__main__":
32     V = 5
33     adj = []
34
35     # creating adjacency list
36     for i in range(V):
37         adj.append([])
38
39     addEdge(adj, 1, 2)
40     addEdge(adj, 1, 0)
41     addEdge(adj, 2, 0)
42     addEdge(adj, 2, 3)
43     addEdge(adj, 2, 4)
44
45     res = bfs(adj)
46
47     for node in res:
48         print(node, end=" ")
```

Output

```
0 1 2 3 4
```

DFS obilazak grafa

- DFS obilazak u dubinu započinje iz proizvoljno izabranog početnog čvora, koji se označava kao posećen, zatim se bira proizvoljni neoznačeni susedni čvor, iz koga se rekurzivno pokreće nova pretraga u dubinu
- Kada se naiđe na čvor koji nema neposećenih suseda, algoritam se vraća na prethodni nivo i nastavlja potragu od sledećeg susednog čvora



DFS počinje od datog izvornog čvora i istražuje jednu putanju što je dublje moguće. Kada stigne do čvora bez neposećenih suseda, vraća se na prethodni čvor kako bi istražila druge neposećene putanje. Ovo se nastavlja sve dok se ne posete svi čvorovi dostupni iz izvora. U grafu mogu postojati petlje. Zato se koristi dodatni niz posećenih čvorova kako bi se osiguralo da se čvor ne posećuje ponovo.

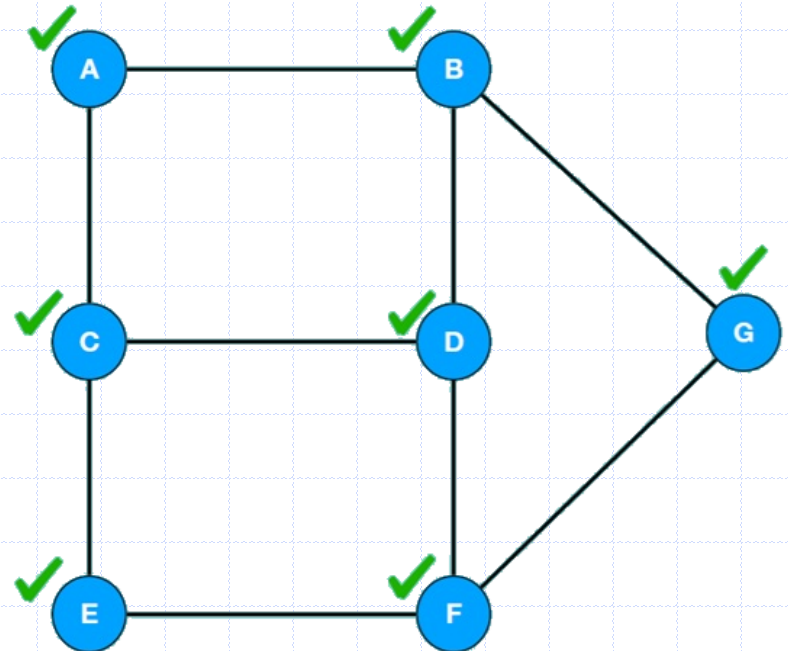
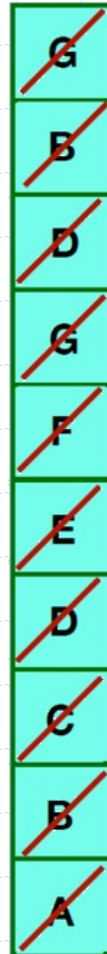
DFS obilazak grafa

DFS

```
push any starting vertex  
while stack is not empty  
  p = pop()  
  if p is unvisited  
    mark it visited  
    Push all adjacent  
    unvisited vertices of p
```

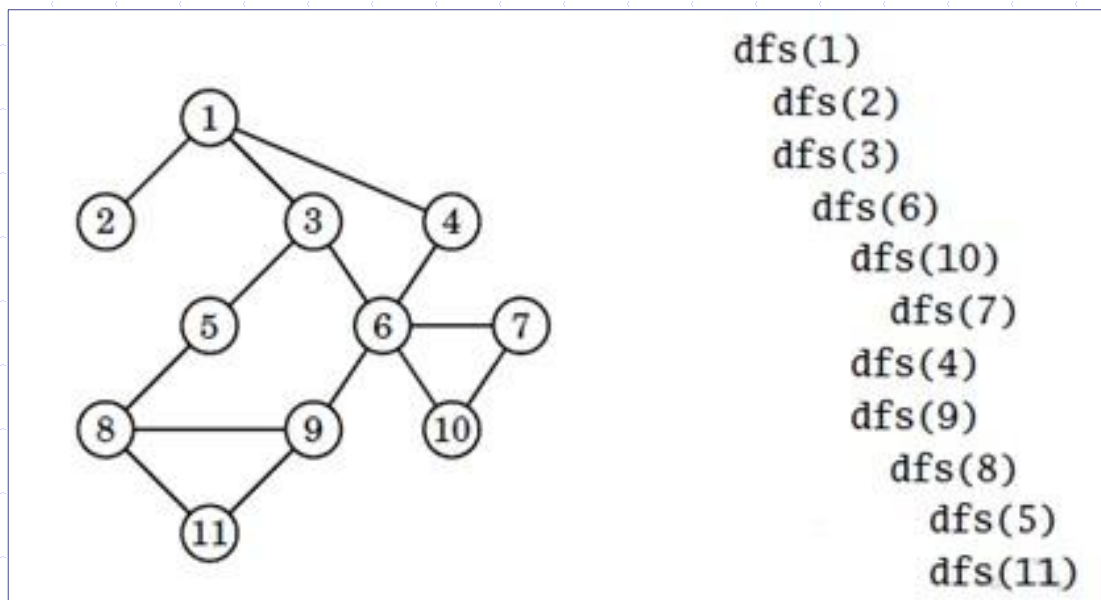
A C E F D B G

Stack



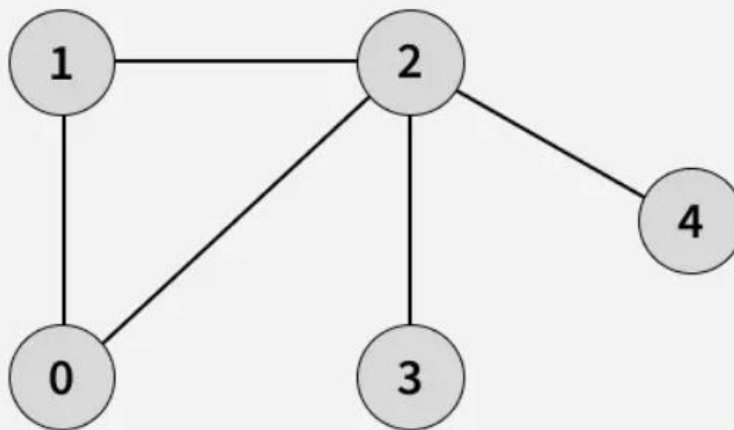
DFS obilazak grafa

- Ako je graf usmeren DFS će nakon završene pretrage obeležiti sve njegove čvorove i proći svaku granu bar jednom
- Kod nepovezanih grafova algoritam se mora pokrenuti više puta iz svakog čvora koji još nije posećen, čime se mogu odrediti sve komponente povezanosti grafa
- Svaki poziv algoritma DFS vrši se samo za neki neposećeni čvor. Algoritam prvo registruje da li je čvor posećen, a zatim se ispituju svi njegovi susedi, ako se pronađe neposećen čvor, rekrzivno se poziva DFS algoritam. Rezultat DFS pretrage nije jednoznačan i zavisi od redosleda kojim se posećuju susedi nekog čvora u **for** petlji algoritma



DFS obilazak grafa

Input: $adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$



Objašnjenje:

Izvorni čvor je 0, prvo se on obilazi, a zatim njegovi susedni čvorovi

Početak od 0: Označiti kao posećeno, ispisati 0

Pomeriti se na 1: Označiti kao posećeno, ispisati 1

Pomeriti se na 2: Označiti kao posećeno, ispisati 2

Pomeriti se na 3: Označiti kao posećeno, ispisati 3 (vratiti se na 2)

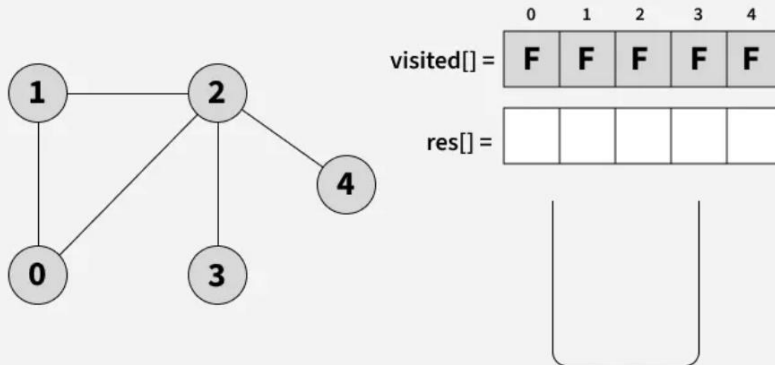
Pomeriti se na 4: Označiti kao posećeno, ispisati 4 (vratiti se na 2, zatim na 1, pa na 0)

Imati na umu da može postojati više od jednog DFS obilaska grafa, npr. posle 1, može se izabrati susedni 2 umesto 0 i dobiti drugačiji DFS

DFS obilazak grafa

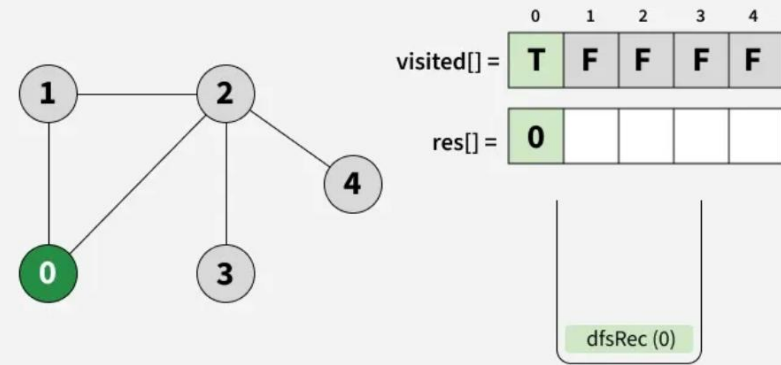
01
Step

Maintain a visited array that tracks which nodes have been explored.



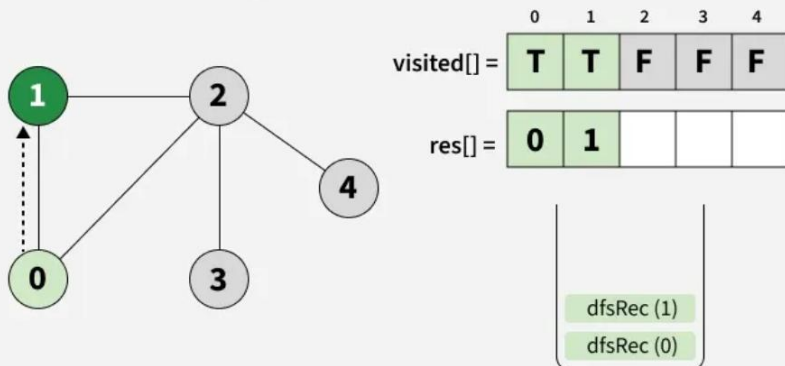
02
Step

We start with source vertex 0.



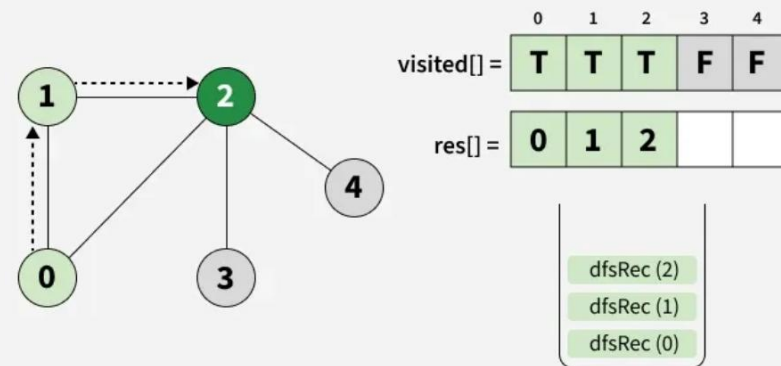
03
Step

We visit the neighbour of 0, i.e., 1.



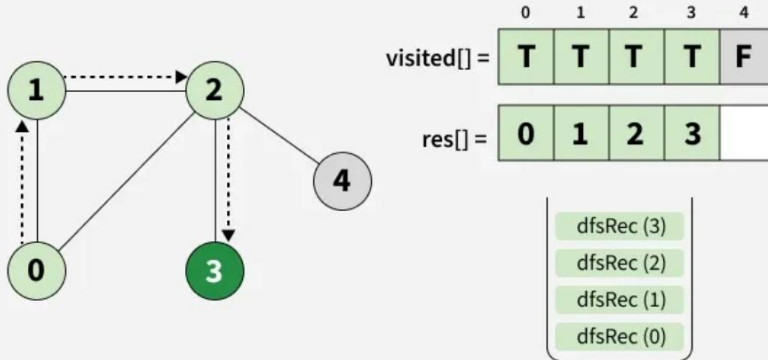
04
Step

We visit the neighbour of 1, i.e., 2.

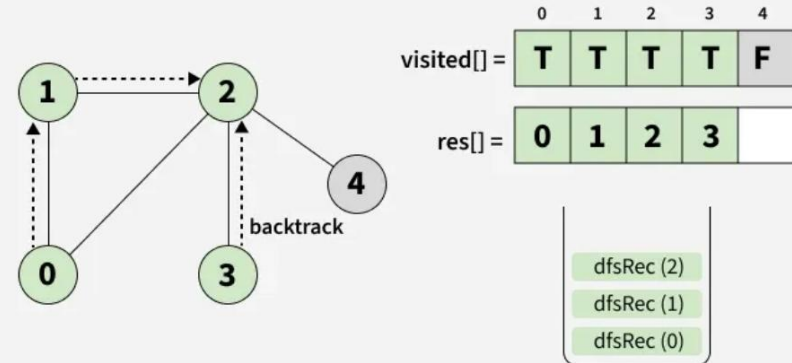


DFS obilazak grafa

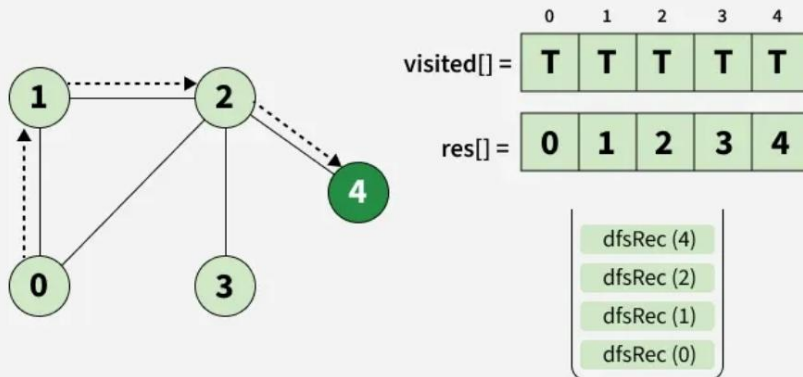
05
Step We visit the neighbour of 2, i.e., 3.



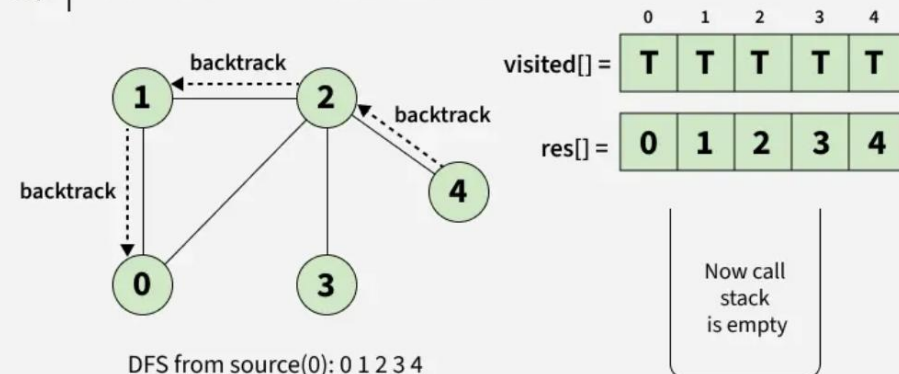
06
Step Backtracking from 3.



07
Step We visit another neighbour of 2, i.e., 4.



08
Step Backtrack all the way to the source node. Since, all the nodes are visited, no further DFS calls will be made.



DFS obilazak grafa – programski kôd

```
1 def dfsRec(adj, visited, s, res):
2     visited[s] = True
3     res.append(s)
4
5     # Rekurzivno obići sve susedne čvorove
6     # koji nisu obuhvaćeni obilaskom
7     for i in adj[s]:
8         if not visited[i]:
9             dfsRec(adj, visited, i, res)
10
11 def dfs(adj):
12     visited = [False] * len(adj)
13     res = []
14     dfsRec(adj, visited, 0, res)
15     return res
16
17 def addEdge(adj, u, v):
18     adj[u].append(v)
19     adj[v].append(u)
20
```

```
21
22 if __name__ == "__main__":
23     V = 5
24     adj = []
25
26     # kreiranje liste susednih čvorova
27     for i in range(V):
28         adj.append([])
29
30     addEdge(adj, 1, 2)
31     addEdge(adj, 1, 0)
32     addEdge(adj, 2, 0)
33     addEdge(adj, 2, 3)
34     addEdge(adj, 2, 4)
35
36     # Izvršavanje DFS počevši od čvora 0
37     res = dfs(adj)
38
39     for node in res:
40         print(node, end=" ")
41
```

Output

```
0 1 2 3 4
```

BFS vs DFS

	BFS	DFS
Način obilaska grafa	Pretraga prvo po širini	Pretraga prvo po dubini
Struktura koja se koristi za obilazak grafa	Red	Stek
Vremenska složenost	$O(V+E)$	$O(V+E)$
Prostorna složenost	$O(V+E)$	$O(V+E)$
Gde se koristi?	Ako znamo da je ciljni čvor blizu početne tačke	Ako znamo da je ciljni čvor duboko u grafu

Primena grafova

- **Društvene mreže:** Predstavljaju korisnike i njihove veze, koriste se za pronalaženje zajedničkih prijatelja, predlaganje novih konekcija i otkrivanje grupa korisnika
- **Računarske mreže:** Modeluju mrežne rutere i prenos podataka, koriste se za efikasno rutiranje, otkrivanje grešaka i optimizaciju rada mreža
- **Transportne mreže:** Predstavljaju gradove i rute (puteve), koriste se za nalaženje najkraćih ili najbržih ruta i planiranje optimalnih ruta putovanja
- **Neuronske mreže:** Predstavljaju neurone i sinapse, koriste se za simulaciju učenja, rada neurona čoveka i njihovu obradu podataka
- **Kompajleri:** Predstavljaju zavisnosti podataka i tokove kontrole izvršavanja programa, koriste se za optimizaciju, alokaciju registara i analizu koda
- **Autonomna vožnja:** Definišu stanja saobraćaja, koriste se za izračunavanje najbezbednije ili najkraće rute za autonomno kretanje vozila
- **Razvoj projekata:** Definišu zadatke i zavisnosti, koriste se u topološkom sortiranju radi određivanja redosleda izvršavanja projektnih zadataka
- **Optimizacija mreža:** Predstavljaju mrežne čvorove i veze, koriste se za minimiziranje troškova, smanjenje latencije i poboljšanje efikasnosti mrežnih i komunikacionih sistema

